Backend Development with FastAPI

Mandatory Requirements

All projects must comply with the following requirements:

1. Project Structure

Students must organize their project using a clean and maintainable structure. The codebase should be modular, properly separated into components such as routes, models, schemas, and services.

2. RESTful API Implementation

The application must implement a fully functional RESTful API, including the following operations:

- **GET**
  - Retrieve all records
  - Retrieve a single record by ID
- **POST**
  - Create new records
- **PUT**
  - Update existing records
- **DELETE**
  - Remove records

Additionally, students must ensure:

- Proper use of **HTTP status codes**
- Request validation using **Pydantic models**
- Use of **response models** for consistent output formatting

3. JWT Authentication

The system must include secure authentication using JSON Web Tokens (JWT), with the following features:

- User registration
- User login
- JWT token generation
- Token validation
- Protection of secured routes using authentication

4. Role-Based Authorization

The application must implement role-based access control with at least two roles, such as:

- **Admin**
- **User**

Access to endpoints must be restricted based on roles. For example:

- Admin users can perform delete operations
- Regular users have limited access (e.g., read-only)

5. Error Handling

The project must include proper error handling mechanisms:

- Use of **HTTPException**
- Clear and descriptive error messages
- Handling of validation errors

Bonus Features (Optional)

API Testing (1 Mark)

- Use **pytest**
- Use **TestClient**
- Test authentication functionality
- Test protected endpoints
- Validate business logic

Docker Integration (1 Mark)

- Create a **Dockerfile**
- Use **docker-compose** (e.g., for FastAPI app and database)
- Ensure the application runs successfully in a Docker environment

Frontend Integration (2 Mark)

- Develop a simple frontend using:
    - HTML / CSS / JavaScript
    - React, Vue, or any other framework
- The frontend must communicate with the FastAPI backend

Project 1: Student Management System

Description:

Develop a backend system to manage university students with secure access control and advanced querying capabilities.

Entities

- Users
- Students

Features

- User registration and authentication
- Full CRUD operations for students
- Advanced filtering (e.g., department, GPA)
- Pagination support
- Students can only access their own profile
- Audit logging for updates

Roles

Admin: Full control over student records
Student: View and partially update own profile

Project 2: Library Management System

Description

Build a backend API for managing a library system with borrowing and tracking functionality.

Entities

- Users
- Books
- BorrowRecords

Features

- CRUD operations for books
- Borrow and return system with availability validation
- Prevent borrowing unavailable books

- Track borrowing history
- Limit number of borrowed books per user

Roles

Admin (Librarian): Manage books and view all records
Member: Borrow, return, and view personal history

Project 3: E-Commerce System

Description

Develop a scalable backend system for an online store with product and order management.

Entities

- Users
- Products
- Categories
- Orders
- OrderItems

Features

- Product and category management (CRUD)
- Product search, filtering, and pagination
- Shopping cart functionality
- Order placement and tracking
- Basic inventory management (stock updates)

Roles

Admin: Manage products and categories, view all orders
Customer: Browse products, manage cart, and place orders

Project 4: Blog Management System

📖 **Description**

Create a backend system for a blogging platform with structured content management.

Entities

- Users
- Posts
- Comments

Features

- CRUD operations for posts and comments
- Ownership-based access control (authors manage their own content)
- Nested comments support
- Pagination for posts and comments

Roles

Admin: Full moderation control
Author: Create and manage own posts
Reader: View and comment

Project 5: Task Management System

Description

Develop a backend system for managing projects and tracking tasks with workflow validation.

Entities

- Users
- Projects
- Tasks

Features

- Project and task management
- Task assignment to users
- Task status lifecycle (To Do → In Progress → Done)
- Enforce valid status transitions

- Filter tasks by status, priority, and assignee

Roles

Admin: Manage projects and tasks
Project Manager: Assign and monitor tasks
Employee: Update assigned tasks only

Project 6: Hospital Appointment System

Description

Build a backend system for managing hospital appointments between patients and doctors.

Entities

- Users
- Doctors
- Patients
- Appointments

Features

- Manage doctors and patients
- Book and cancel appointments
- Prevent double booking
- View schedules by doctor and patient
- Appointment status management (Scheduled, Completed, Cancelled)

Roles

Admin: Manage doctors and patients
Doctor: View and update appointment status
Patient: Book and manage appointments

Project 7: Online Examination System

Description

Create a backend system for conducting and managing online examinations.

Entities

- Users
- Exams
- Questions
- StudentAnswers
- Results

Features

- Create and manage exams and questions
- Support timed exams
- Automatic grading for objective questions
- Store and retrieve results
- Basic analytics (scores and averages)

Roles

Admin: Manage exams and view all results
Student: Take exams and view results